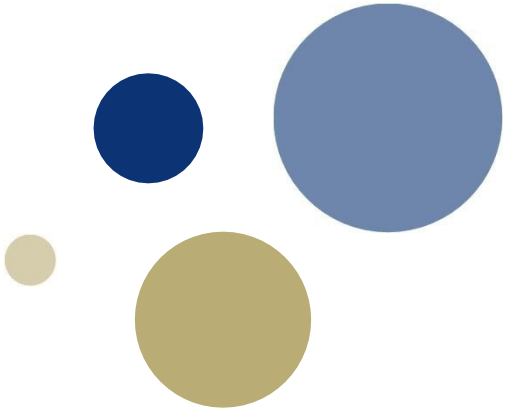
NTNU

Kunnskap for en bedre verden



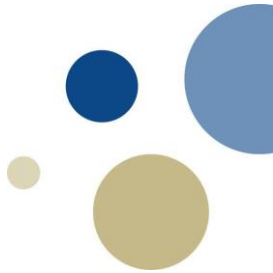
Python: Mengder og Ordbøker

TDT4110 IT Grunnkurs

Professor Guttorm Sindre,

Dag Olav Kjellemo

Læringsmål og pensum

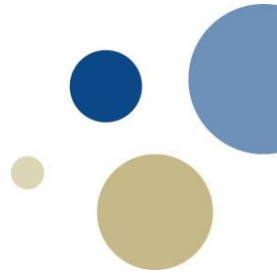


- Mål
  - Forstå nytten av hashede datastrukturer
    - Vite forskjell på disse og lister, tupler
  - Forstå prinsippene for, og kunne bruke i praksis
    - Mengder (sets)
    - Ordbøker / oppslagslister (dictionary)

Allerede lært

- Sammensatte variable: inneholde flere verdier
  - lister, f.eks. L = [2, 5, 4, 7, 6, 9]
  - tupler, f.eks. T = (2.5, 7.6, 3.0)
- Begge (samt også strenger) er sekvenser
  - Enkeltelementer kan finnes ved bruk av heltallsindeks, f.eks.
    - L[0] -> 2
    - T[2] -> 3.0
  - Indeksen baserer seg på posisjon (rekkefølge av data)
- I sekvenser er det kjapt å
  - Finne hvilken verdi som står på indeks i
  - Legge til / fjerne element bakerst (hvis muterbar)
- ...men tregere å...
  - Slå opp på verdier direkte
  - Legge til / fjerne element annet sted enn bakerst

Ofte har vi behov for

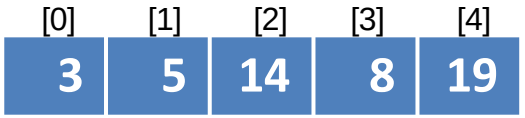


- Hyppige oppslag på verdi
- Forholdsvis hyppig innsetting og sletting av data
  - Ikke nødvendigvis «bakerst»
- Ønsker datatyper som gir direkte oppslag på verdi
  - I motsetning til lister, tupler som har direkte oppslag på indeks

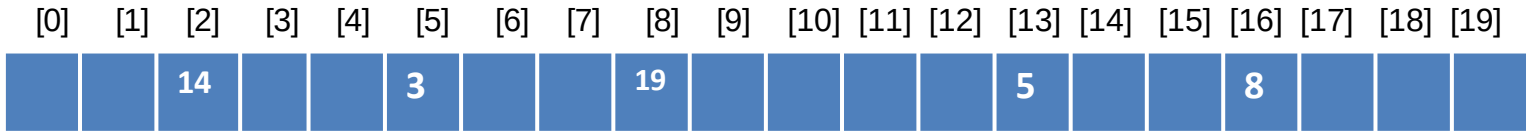
- En løsning: *Hashede* datatyper
  - i Python / vårt pensum:
    - mengder (sets),
    - ordbøker (dictionary)

Veldig kort om hashing

- Liste, L = [3, 5, 14, 8, 19]
- Verdiene legges etter hverandre i minnet



- Mengde, S = {3, 5, 14, 8, 19}
- Verdier kjøres gjennom en hashfunksjon h(x)
  - Mål: hver verdi havner på en unik indeks, f.eks. h(14) -> 2



  - Senere: Lete etter 14? Regne ut h(14), slå opp på indeks 2
- Bruke ekstra plass, lav sannsynlighet for kollisjon

Sammenligning, plassbruk

Kode: plass_og_tid.py

- Oppretter liste, tuppel, mengde med 10,000 element

```
L = list(range(0,30001,3))  # [0,3,6,...,30000]
T = tuple(range(0,30001,3)) # (0,3,6,...,30000)
S = set(range(0,30001,3))   # {0,3,6,...,30000}
```

- Sammenligning, plassbruk i minnet

```
print("Plassforbruk, variable")
import sys #har en funksjon for plassforbruk
print("L", sys.getsizeof(L))
print("T", sys.getsizeof(T))
print("S", sys.getsizeof(S))
```

Mengde: ca. 6x plassbruk…

```
Plassforbruk, variable
L 45064
T 40032
S 262260
```

Sammenligning tidsbruk

```
print("Tidsforbruk, oppslag på verdi")
import timeit #har en funksjon for tidtaking
print("L", timeit.timeit("15000 in L", number=10000, globals=globals()))
print("T", timeit.timeit("15000 in T", number=10000, globals=globals()))
print("S", timeit.timeit("15000 in S", number=10000, globals=globals()))
```

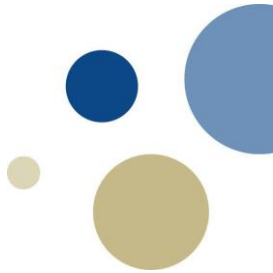
```
Tidsforbruk, oppslag på verdi
L 0.5225763
T 0.5170834
S 0.0005475999999999814
```

Liste, tuppel: ca. et halvt sekund

Mengde: ca. et halvt *mikro*sekund

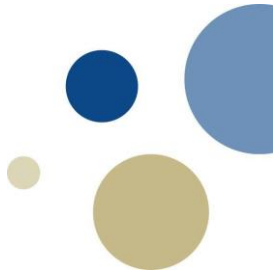
Mengde 1000x raskere!

Læringsmål og pensum



- Mål
  - Forstå nytten av hashede datastrukturer
    - Vite forskjell på disse og lister, tupler
  - Forstå prinsippene for, og kunne bruke i praksis
    - Mengder (sets)
    - Ordbøker / oppslagslister (dictionary)
- Pensum
  - Starting out with Python, Chapter 9:
    - Dictionaries and Sets

Set – mengder



- Datastruktur for matematiske mengder
  - Elementer trenger ikke være tall, også andre ikke-muterbare datatyper
- Unike element, ingen dubletter
- Ikke ordnet, dvs. ingen bestemt rekkefølge,
  - Dvs. vi har ingen kontroll på rekkefølgen
- De fleste funksjoner for lister kan brukes for mengder
- Mengder har flere spesifikke operasjoner, bl.a.:
  - *union*
  - *intersection* = norsk: snitt
  - Ulike typer mengdedifferanser

Opprette og endre på mengder

- Mengde opprettes med funksjonen set( )

```
a = set()                 # gir tom mengde
                          # ikke a = {} (tom dictionary)

b = set([5,3,3,12])       # gir {5, 3, 12}

c = set("aaabcd")         # gir  {"a","b","c","d"}
```

- Eller direkte angi innhold:

```
d = {9,4,1,3}
```

- Legge til og fjerne element:

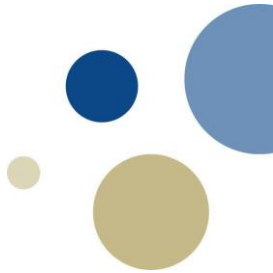
```
b.add(9)                  # gir {5, 3, 12, 9}

b.update([3,4,2])         # gir {5, 3, 12, 9, 4, 2}

b.remove(3)     # gir {5, 12, 9, 4, 2},  NB! Må inneholde

b.discard(10)             # gir ikke feilmelding
```

Hvorfor ikke bare lister?



Du har et dataprogram registrerer påmeldinger.

Påmeldte er lagret som en liste

```
deltagere = [‘Ada’, ‘Even’, ‘Per’]
```

Even melder seg på nytt (glemsk som han er). Hvis vi bare legger ham til, så får vi

```
deltagere.append(nyDeltager)    #  [‘Ada’, ‘Even’, ‘Per’, ‘Even’]
```

Even melder seg så av (ubestemt også!):

```
deltagere.remove(‘Even’)        #   [‘Ada’, ‘Per’, ‘Even’]
```

Even er fortsatt med!

Løsningen er å sjekke om Even allerede er med før vi legger ham til:

```
If nyDeltager not in deltagere:
    deltagere.append(nyDeltager)    #  [‘Ada’, ‘Even’, ‘Per’]
```

Hvorfor ikke bare lister?

For å finne en element i en liste, eller å finne ut om det i det hele tall er i en liste, så må Python gå igjennom listen element for element, og sammenligne med det vi leter etter.
Dette gjelder både for in-operatoren for lister, og index-metoden

En kan selfølgelig fikse dette ved å sjekke om en deltager allerede er registrert.

Men det er relativt tregt å søke gjennom lister.

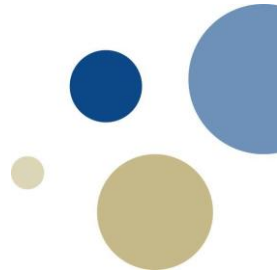
Hvorfor ikke bare lister?

Selv om vi relativt lett kan håndtere medlemskap med lister, så er det *mengder* som er det riktige *konseptet*.
Ved å bruke riktige konsepter, kan en lettere resonnere og lage løsninger, Krever mindre koding, og vil ofte gi mer effektiv kode.

For mengder kan legge til uten fare for duplikater. Ingen feilmelding heller.

In-operatoren er rask når brukt på mengder.

Iterasjon og medlemsskap



- For-løkke kan iterere over en mengde
  - Som for lister, tupler, strenger
  - Men ingen garanti for rekkefølgen vi får elementene i

```
for x in mengde:
    print(x)          # Skriver ut elementer under hverandre
```

- Undersøke om verdi tilhører mengde: **in** , **not in**
  - Tilsvarer $\in$ og $\notin$ i matematisk notasjon

```
if x in A:
    print(f"Verdien {x} er i mengde A")
```

Mengdeoperasjoner på A og B

- Anta A = {1, 2, 3, 4} ; B = {2, 4, 6, 7}
- Union av to mengder:

```
C = A.union(B)                  # C = A U B, dvs. {1,2,3,4,6,7}
```

- Snitt av to mengder:

```
C = A.intersection(B)           # C = A ∩ B, dvs. {2, 4}
```

- Mengdedifferanse:

```
C = A.difference(B)             # C = A \ B, dvs. {1, 3}
```

- Symmetrisk mengdedifferanse:

```
C = A.symmetric_difference(B)   #  C = (A U B) \ (A ∩ B), dvs. {1,3,6,7}
```

- Sjekke delmengde eller supermengde:

```
A.issubset(B)                   # True hvis A ⊂ B (men her False)
A.issuperset(B)                 # True hvis A ⊃ B, (men her False)
```

Mengdeoperasjoner på A og B

- Anta A = {1, 2, 3, 4} ; B = {2, 4, 6, 7}
- Vi har også varianter med “update”, som endrer Mengen “in place”:
- Snitt av to mengder:

```
A.intersection_update(B)          # A = A ∩ B, dvs. {2, 4}
```

- Mengdedifferanse:

```
A.difference_update(B)            # A = A \ B, dvs. {1, 3}
```

- Symmetrisk mengdedifferanse:

```
A.symmetric_difference_update(B)  #  A = (A U B) \ (A ∩ B), dvs. {1,3,6,7}
```

Oppgave: Lotto

Skriv kode for funksjonen

- **sjekk_rekke**(mine_tall, riktige_tall)

som sjekker hvor mange av mine_tall som er rette.

Som trening på andre ting vi har gjort, skriv kode for funksjonene

- **trekk_tall**(antall, maks) som genererer antall forskjellige tall mellom 1 og maks.
- **velg_tall**(antall, maks) som ber brukeren om antall tall mellom 1 og maks.

Lag så en funksjon som kjører en lotto-trekning og ber brukeren om tall, og så printer antall rette.

lotto.ipynb

Oppgave: Kake-mysteriet

- Vi ønsker et program som gjør dette:

```
# Hvem har spist opp kaken?
- Kaken stod i kjøleskapet på kjøkkenet.
- Adam, Berit, Carl, Dorte, Erik, Fride hadde tilgang
- Arne, Berit, Cecilie, Dorte, Erik, Fiona liker kake.
- Berit og Erik hadde alibi.

Hvem spiste opp kaken?
```

- Lag funksjonene
  - lag_mengde() – streng som tastes inn -> mengde med navn
    - Hint: strengmetoden split() og funksjonen set()
  - finn_mistenkte() – 3 mengder -> mengden av navn som
    - har tilgang og motiv (hint: metoden intersection)
    - men IKKE har alibi (hint: metoden difference)

Kakemysteriet.ipynb

Oppgave («VANSKELIG»)

Gitt ei 2D liste hvor hver indre liste forteller hvilke deltagere i et turløp som har vært på et visst fjell. (I dette eksemplet bare 3 fjell)

```
fjell_lister =  [['Jo', 'Ine', 'Eli', 'Bo', 'Ron', 'Sam', 'Una', 'Ron'],
                 ['Eli', 'Ada', 'Oda', 'Jo', 'Dag', 'Una', 'Ron'],
                 ['Bo', 'Ada', 'Tor', 'Dag', 'Jo', 'Eli', 'Frank']]
```

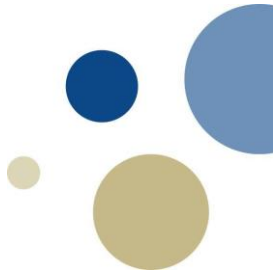
Vi vil ha et program som kan liste opp

- Alle som har deltatt (vært på minst ett fjell)
- De som har vært på *alle* fjellene

Lag funksjoner deltatt() og alle_fjell()

```
Alle deltagere: {'Ron', 'Sam', 'Una', 'Ada', 'Bo', 'Frank',
'Eli', 'Ine', 'Tor', 'Dag', 'Oda', 'Jo'}
Alle 3 fjell: {'Eli', 'Jo'}
```

Læringsmål og pensum



- Mål
  - Forstå nytten av hashede datastrukturer
    - Vite forskjell på disse og lister, tupler
  - Forstå prinsippene for, og kunne bruke i praksis
    - Mengder (sets)
    - Ordbøker / oppslagslister (dictionary)
- Pensum
  - Starting out with Python, Chapter 9:
    - Dictionaries and Sets

Ordbøker (Dictionary)

- Lagrer en samling av data.
- Som for mengder: direkte oppslag på nøkkelverdi
- Dessuten: **Andre data i tillegg til nøkkelverdiene**
  - En mengde av par: **{nøkkel : verdi, nøkkel : verdi, ...}**

- Illustrasjon av forskjell på mengde og ordbok:

```
grunnstoff_mengde = {'H','He','Li','Be','B','C','N','O'}
                    # osv. med flere grunnstoff

grunnstoff_ordbok = {'H': 'Hydrogen', 'He': 'Helium',
                     'Li': 'Litium', 'Be': 'Beryllium',
                     'B': 'Bor', 'C': 'Karbon',
                     'N': 'Nitrogen', 'O': 'Oksygen'}
                    # osv. med flere grunnstoff
```

Forskjell på ordbok og 2D-liste

- Verdi-delen av **nøkkel : verdi** kan f.eks. være ei liste
  - Eksempel: Grunnstoff hvor forkortelse er nøkkel
  - Verdi er liste med navn, atomnummer, atomvekt

```
ordbok = {'H': ['Hydrogen', 1, 1.008],
          'He': ['Helium', 2, 4.003],
          'Li': ['Litium', 3, 6.939]}
```

Forskjell på ordbok og 2D-liste

- Verdi-delen av **nøkkel** **: verdi** kan f.eks. være ei liste
  - Eksempel: Grunnstoff hvor forkortelse er nøkkel
  - Verdi er liste med navn, atomnummer, atomvekt

```
ordbok = {'H': ['Hydrogen', 1, 1.008],
          'He': ['Helium', 2, 4.003],
          'Li': ['Litium', 3, 6.939]}
```

Forskjell på ordbok og 2D-liste

- Verdi-delen av **nøkkel : verdi** kan f.eks. være ei liste
  - Eksempel: Grunnstoff hvor forkortelse er nøkkel
  - Verdi er liste med navn, atomnummer, atomvekt

```
ordbok = {'H': ['Hydrogen', 1, 1.008] ,
          'He': ['Helium', 2, 4.003] ,
          'Li': ['Litium', 3, 6.939] }
```

Forskjell på ordbok og 2D-liste

- 2D-liste kan inneholde samme info som ordbok m. liste
- 2D-liste, raskt oppslag på indeks, men ikke på verdi

```
ordbok = {'H': ['Hydrogen', 1, 1.008],
          'He': ['Helium', 2, 4.003],
          'Li': ['Litium', 3, 6.939]}

liste2 = [[ 'H', 'Hydrogen', 1, 1.008],
          ['He', 'Helium', 2, 4.003],
          ['Li', 'Litium', 3, 6.939] ]
```

Opprette og endre ordbok



```
grunnstoff = { }  # oppretter tom ordbok
grunnstoff = {'H': 'Hydrogen', 'He': 'Helium'}
# opprettet med verdier
grunnstoff['Li'] = 'Litium'
# lagt til nytt nøkkel:verdi-par
grunnstoff['H'] = 'Vannstoff'
# endret verdi for nøkkelen 'H'

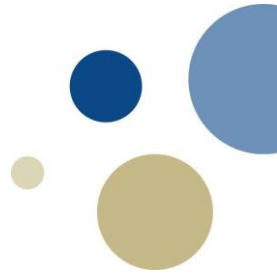
grunnstoff.pop('He')
# fjernet paret 'He': 'Helium'
```

Bruk av in og not in

- Sjekke om elementer finnes
  - som lister, mengder etc.

```
tlf={'Jo' : 73540000, 'Per' : 92542312, 'Else' : 54239212}
navn = input('Hvem vil du vite nummeret til? ')
if navn in tlf:
    print(tlf[navn])
else:
    print('Har ingen opplysninger om' , navn)
```

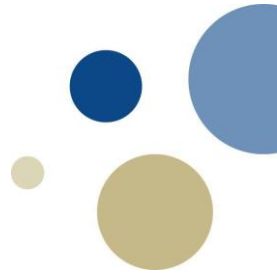
Operasjoner for ordbøker



Operasjon	Forklaring
len(d)	Antall elementer i d
d[k]	Verdi til element i d med nøkkel k
d[k] = v	Sett element k til verdi v
del d[k]	Slett element k i d
d.clear()	Fjerne alle element i d
d.copy()	Lag kopi av d
~~d.has_key(k)~~	Fjernet i Python 3. Må bruke **in** i stedet.

Operasjon	Forklaring
d.items()	Returnerer liste av (nøkkel,verdi) par
d.keys()	Returnerer liste av nøkler i d
d.values()	Returnerer liste av verdier i d
d.get(k)	Samme som d[k]
d.get(k,v)	Returnerer d[k] hvis k er gyldig, ellers v

for-løkke med ordbøker



- for-løkke på en ordbok itererer gjennom *nøklene* :

```
d = {'H' : 'Hydrogen', 'He' : 'Helium', 'Li' : 'Litium'}
for x in d:                    # går igjennom nøkler i d
    print(x)                   # skriver ut H, He, Li
    print(d[x])                # skriver ut Hydrogen, Helium, Litium
```

- Sortering kan brukes for å oppnå en viss rekkefølge:

```
for x in sorted(d.keys(), reverse = True):
    print(x)                   # skriver ut Li, He, H
```

Hva bruke som nøkkel?

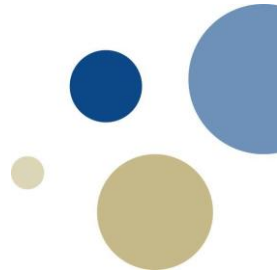
- Kommer an på behovet, hva vi vil slå opp på
  - Eks.: Persondata
    - Personnummer.
      - Fordel: unikt. Ulempe: husker det ikke, bør ikke spres for mye
    - Telefonnummer… navn…
  - Eks.: liste med ord
    - Oversettelse norsk-engelsk: { ’god’: ’good’, ’stor’: ’big’ }
    - Gradbøying: { ’god’: [’bedre’, ’best’], ’stor’: [’større’, ’størst’] }
    - Rimkatalog: {’erte’: [’berte’, ’erte’, ’fjerte’, ’hjerte’, ’smerte’, ’terte’] }
    - Anagrammer: finne ord med samme bokstaver uavh. av rekkefølge
      - {’ad’, : [’ad’, ’da’], ’ert’: [’ert’, ’ter’, ’tre’] , ’eertt’ : [’etter’, ’rette’, ’treet’] …}

Programmeringsoppgave

- LETT: Skriv et program som
  - Oppretter ei ordbok tlf med elementene
    - ‘Per’ : 95959595
    - ‘Ida’ : 90919293
  - Legger til elementet ‘Ine’ : 94939291
  - Bruker ei for-løkke til å printe ut:

```
>>> %Run lett.py
Jo 95959595
Ida 90919293
Ine 94939291
```

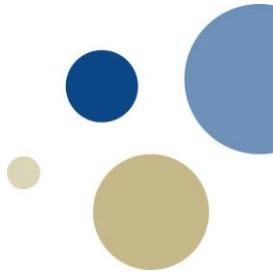
Serialisering av objekter



- Konverterer et objekt til en strøm av bytes
  - Gir en persistent variabel
  - Data lagres til fil, kan senere leses tilbake
  - Verdiene overlever programslutt, at maskin slås av, etc.
  - I Python: biblioteket *pickle* har funksjoner for dette
- Fordeler med binærfiler (vs. tekstfil)
  - Får dataene direkte i ønsket datastruktur, f.eks. ordbok
  - Slipper konvertering til / fra strenger
  - Vanligvis mindre plasskrevende

NB! Kan være en sikkerhetsrisiko å bruke siden den utfører kode ved innlesning, som kan utnyttes: Dvs. ikke bruk Pickle til å lese filer fra ukjente kilder

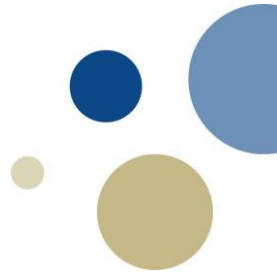
Lagre ordbok til disk



- Bruk biblioteket pickle
  - metoden pickle.dump( )
- Må åpne fila som binærfil og ikke som tekst,
  - det er mer enn tekst som lagres (struktur):

```
gr_data={ 'H' : ['Hydrogen', 1, 1.008], 'He' : ...}
import pickle                          # Importerer modulen
f = open( 'grstoff.dat' , 'wb' )       # b=binary
pickle.dump(gr_data ,f)                # Lagrer data til disk
f.close()                              # Lukker fila
```

Laste inn ordbok fra disk



- Metoden pickle.load()
- Husk å åpne fila som binærfil og ikke tekst.

```
import pickle
f = open( 'datafil.dat' , 'rb' )     # r=read, b=binary
data = pickle.load(f)                # Laster inn dict fra disk
f.close()
```

Oppsummering

- Mengder (sets) er nyttige for å gjøre operasjoner som
  - Snitt, union og mengdedifferanse
  - Mengder defineres ved å bruke funksjonen **set(liste)**
- Oppslagsliste (dictionary) ligner på lister, men…
  - En kan bruke hva som helst ikke-muterbart som nøkkel (indeks).
  - Kan opprettes tom: **A = {}, eller A = dict()**
  - Eller ved nøkkel + data: **A = {'navn': 'Petter Ole'}**
  - Trenger ikke indeks som er i rekkefølge
  - Kan serialiseres / dumpes til binærfil